01-architecture.md architecture[1]

ProjectEnhancer — Architecture

Document type: Product architecture baseline
Status: v1.0
Purpose: Define the safe, multi-language architecture for creating an approved Enhanced Version 2 without directly changing the user's original project.

1. Product promise

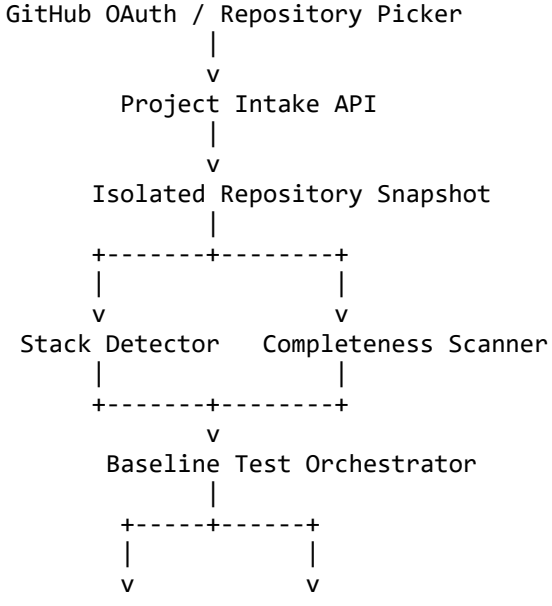
ProjectEnhancer is a review-first platform that connects to a user's GitHub repositories, detects the real technology stack, checks completeness and baseline health, repairs only approved existing issues, and recommends enhancements for already-created project areas.

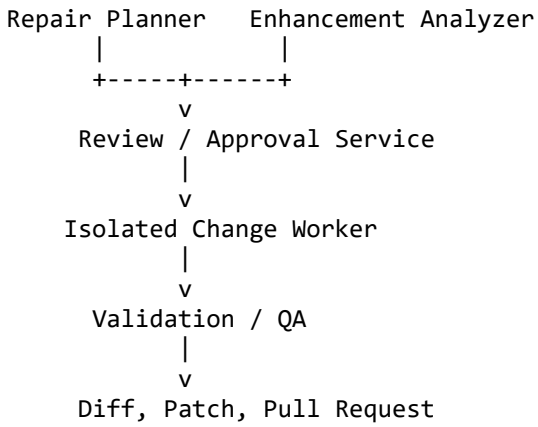
The original repository is never silently modified. All repairs and enhancements occur in an isolated workspace and a separate branch or patch.

2. Non-negotiable principles

1. **Original project safety:** never write directly to `main`/`master`.
2. **Evidence before inference:** do not hallucinate languages, frameworks, files, features, or test results.
3. **Completeness before testing:** distinguish working, broken, partial, placeholder, and not implemented.
4. **Repair before enhancement:** pre-existing errors are separated from Version 2 improvements.
5. **Existing scope only by default:** in Limited Enhancement Mode, modify only areas that already exist.
6. **User approval:** every repair and enhancement is reviewable and explicitly approved.
7. **Traceability:** record base commit, files, components, elements, risk, tests, and approvals.
8. **Responsive single viewport:** app screens show one primary task; use wizards, tabs, routes, or pagination instead of clipping.

3. High-level architecture





4. Core modules

Repository integration

- GitHub OAuth and scoped repository access.
- Repository and branch selection.
- Immutable base commit capture.
- Read-only snapshot before any change.

Stack detection

Evidence comes from committed files such as `package.json`, `pyproject.toml`, `pom.xml`, `go.mod`, `Cargo.toml`, `.csproj`, `composer.json`, `pubspec.yaml`, and build/test configuration. Unknown values remain **Not detected**.

Completeness scanner

Finds TODO/FIXME markers, empty files, placeholders, missing routes, dummy data, unimplemented modules, broken imports, missing configuration, and partial feature areas. It classifies each area as **Working**, **Working with issues**, **Broken**, **Partial**, **Placeholder**, or **Not implemented**.

Baseline and repair engine

Runs available dependency, build, unit, integration, functional, system/runtime, lint, typecheck, and security checks. A missing test suite is reported as **Not available**, not as a failure. Approved repairs run on a repair branch and must be validated again.

Enhancement engine

Analyzes only applicable and existing areas for UI/UX, architecture, security, performance, testing, documentation, and design. Missing features are excluded in Limited Enhancement Mode.

Change and review engine

Produces file-level, component-level, element-level, folder-level, and line-level impact. Each recommendation has a risk level, expected behavior impact, affected files, and an approval state.

5. Branch and version model

```
main                              Original; never directly changed
  |
  +-- ai/repair/<run-id>           Approved baseline repairs only
  |
  +-- ai/enhancement/<run-id>      Approved Version 2 enhancements
```

The platform stores the original commit and creates a change manifest. The user decides whether to merge a pull request.

6. Multi-language adapter model

The platform has a language-neutral core and adapters for JavaScript/TypeScript, Python, Java, Go, PHP, C#/.NET, Rust, Dart/Flutter, Ruby, and other supported stacks. An adapter defines detection evidence, package manager, build command, test command, lint/typecheck command, runtime checks, and safe refactoring rules.

Unsupported or ambiguous stacks receive analysis-only treatment until the user confirms the technology and a safe adapter exists.

7. Runtime safety

Repository code runs in isolated, resource-limited workers. Network, CPU, memory, disk, process count, and execution time are constrained. Secrets are detected and excluded from unnecessary logs or model context. No repository script runs directly on the application server.

8. Acceptance criteria

- Original branch remains byte-for-byte unchanged by the platform.
- Every applied change maps to an approved recommendation or repair item.
- No unknown technology is presented as fact.
- Partial projects are not falsely marked as broken or complete.
- Failed validation blocks the next phase.
- A final diff and validation receipt are available before pull request creation.